Hands-on Lab: Tìm hiểu React qua dự án react_research

1. Mục tiêu và cách học

Tài liệu hướng dẫn bạn chạy dự án trên máy cá nhân và thực hiện các bài nhỏ để học React qua ứng dụng tra cứu phim. Bạn sử dụng mã nguồn có sẵn, đọc các phần liên quan và thay đổi từng bước; không cần xây dựng lại toàn bộ ứng dụng từ đầu.

Sau khi hoàn thành, bạn có thể giải thích component, JSX, props, state, custom Hook; theo dõi luồng dữ liệu từ API đến giao diện; sử dụng định tuyến; quan sát tải mã theo trang và xử lý lỗi; tạo bản build và thực hiện phép so sánh Compiler cơ bản.

Kiến thức đầu vào: HTML, CSS và JavaScript cơ bản, đặc biệt là hàm, mảng, `map`, `import/export` và `async/await`. Nếu mới tiếp cận React, đọc React Quick Start song song với bài thực hành.

Lộ trình: phần 2–3 để chạy nhanh; phần 4–9 để học qua thao tác; phần 10–11 để build và tìm hiểu Compiler. Có thể chia thành nhiều buổi. Sau mỗi bài, ghi lại thay đổi, kết quả quan sát và câu trả lời tự kiểm tra.

2. Chuẩn bị và lấy mã nguồn

Chuẩn bị Node.js **nhánh 24, từ phiên bản 24.15.0 trở lên trong nhánh này**, kèm npm; trình soạn thảo như VS Code; trình duyệt có Developer Tools; Internet để cài dependency và gọi API. Cần Git nếu clone mã nguồn, tài khoản TMDB và API Read Access Token cho các bài dùng dữ liệu thật.

Mốc Node trên được chọn theo trường `engines` của các package trong `package-lock.json` hiện tại, bao gồm Vite, Vitest và jsdom. Chỉ đáp ứng mức tối thiểu của Vite chưa chắc đủ cho bộ kiểm thử. Tham khảo hướng dẫn cài đặt Vite.

Kiểm tra trong terminal:

```
node --version
npm --version
```

Nếu chưa có mã nguồn, lấy dự án từ kho mã:

```
git clone https://github.com/Huy-NTG/react_research.git react_research
cd react_research
npm ci
```

Nếu kho mã yêu cầu quyền truy cập, đăng nhập bằng tài khoản được nhóm cấp quyền hoặc sử dụng bản ZIP do nhóm cung cấp. Nếu đã có dự án hoặc nhận tệp ZIP, mở terminal tại thư mục chứa `package.json` rồi chạy `npm ci`. Lệnh này cài phiên bản đã chốt trong `package-lock.json`; không cần chạy `npm create vite` hay cài React riêng. Khi nhóm chỉ định nhánh hoặc mã commit cho buổi học, chuyển về phiên bản đó trước khi cài thư viện.

Kết quả mong đợi: cài đặt hoàn tất, xuất hiện `node_modules`. Nếu gặp lỗi, xem phần 12.

3. Chạy ứng dụng lần đầu

3.1. Cấu hình TMDB

Sao chép `.env.example` thành `.env` ở thư mục gốc.

macOS/Linux:

```
cp .env.example .env
```

Windows PowerShell:

```
Copy-Item .env.example .env
```

Đăng nhập TMDB, vào phần cấu hình API và lấy **API Read Access Token** theo hướng dẫn xác thực ứng dụng TMDB. Điền vào `.env`:

```
VITE_TMDB_READ_ACCESS_TOKEN=TOKEN_THUC_TE_CUA_BAN
VITE_TMDB_ACCOUNT_ID=ID_TAI_KHOAN_CUA_BAN
```

Chỉ dán giá trị token, không thêm `Bearer` vì mã nguồn tự thêm tiền tố này. Không nhầm API Read Access Token với API Key. Account ID là ID dạng số, không phải tên đăng nhập; chỉ cần cho chức năng tài khoản. Khi chưa có ID, xóa dòng tương ứng thay vì giữ chuỗi ví dụ. Mã có ID mặc định, nhưng không nên dùng ID đó để thực hiện thao tác ghi.

Chưa có token: để `VITE_TMDB_READ_ACCESS_TOKEN=` trống. Trang chủ có dữ liệu mẫu ở mục Trending, nhưng tìm kiếm, chi tiết, phim phổ biến và trailer chưa hoạt động đầy đủ; poster mẫu cũng có thể không tải được. Không giữ chuỗi `your_tmdb_read_access_token` vì ứng dụng sẽ xem đó là token đã cấu hình.

Chức năng tài khoản là bài tùy chọn: thêm/xóa watchlist và đánh dấu yêu thích còn phụ thuộc xác thực người dùng và quyền API. Dự án chưa có luồng đăng nhập TMDB hoàn chỉnh và chưa tự tạo `session_id`; token đọc và account ID không tự bảo đảm thao tác ghi thành công. Xem xác thực người dùng TMDB; bỏ qua thao tác ghi khi chưa có quyền phù hợp.

Tệp `.env` đã được loại khỏi Git. Tuy nhiên, biến có tiền tố `VITE_` được đưa vào mã trình duyệt và không phải nơi lưu bí mật phía máy chủ. Dùng thông tin dành cho bản thực hành và rà soát cách cấp quyền trước khi chia sẻ bản triển khai. Tham khảo biến môi trường Vite.

3.2. Khởi động và dùng thử

```
npm run dev
```

Mở URL `Local` trong terminal, thường là `http://localhost:5173`; nếu cổng bận, Vite có thể chọn cổng khác. Dừng bằng `Ctrl+C`. Sau khi sửa `.env`, dừng rồi khởi động lại.

Với token đọc hợp lệ, thử lần lượt:

1. Xem Trending, chuyển `Today` / `This Week`.
2. Nhập `Batman` vào ô tìm kiếm, chờ khoảng 400 ms để điều hướng.
3. Chuyển `Movies` / `TV Shows`, chuyển trang nếu có kết quả.
4. Về trang chủ, nhấn thẻ phim để xem chi tiết.
5. Mở trailer nếu dữ liệu có video tương ứng.

Kết quả mong đợi: dữ liệu hiển thị và chuyển được giữa các màn hình chính. Một số menu như hồ sơ, đánh giá, cài đặt đang dẫn tới trang tiếp tục phát triển. Hiện tại, mở chi tiết từ thẻ phim trang chủ thay vì kỳ vọng mọi kết quả tìm kiếm đã có liên kết chi tiết.

4. Bài 1 — Đọc cấu trúc ứng dụng

Mục tiêu: biết bắt đầu đọc mã từ đâu, phân biệt React và thư viện hỗ trợ.

Tệp/thư mục	Vai trò
`src/main.jsx`	Gắn ứng dụng vào DOM, cung cấp Router và QueryClient
`src/App.jsx`	Gọi bộ định tuyến
`src/routes/AppRouter.jsx`	Đường dẫn, Suspense và Error Boundary
`src/routes/Lazypages.jsx`	Khai báo các trang dùng `lazy`
`src/pages/`	Các màn hình chính
`src/components/`	Những phần giao diện được chia nhỏ
`src/hooks/`	Truy vấn dữ liệu và thao tác nghiệp vụ
`vite.config.js`	Build, Compiler, phân tích bundle và cấu hình test

Đọc theo thứ tự `main.jsx` → `App.jsx` → `AppRouter.jsx` → `HomePage.jsx` → `TrendingSection.jsx` → `useTrendingMovies.js` → `MovieCard.jsx`.

Vẽ luồng: chọn khoảng thời gian → cập nhật state `period` → Hook truy vấn theo khoảng thời gian → dữ liệu truyền vào thẻ phim qua props → hiển thị danh sách.

Tự kiểm tra: component nào giữ `period`? Tệp nào gọi API? Vì sao `MovieCard` không cần tự gọi API? React Router và TanStack Query bổ sung chức năng gì?

5. Bài 2 — Component, JSX và props

Mục tiêu: truyền nội dung từ component cha xuống component con.

1. Mở `src/components/HeroBanner/HeroBanner.jsx`, đổi khai báo hàm thành:

```
function HeroBanner({ title = 'Welcome' }) {
```

2. Thay dòng tiêu đề bằng:

```
<h1>{title}<span>.</span></h1>
```

3. Trong `src/pages/HomePage/HomePage.jsx`, thay `<HeroBanner />` bằng:

```
<HeroBanner title="Khám phá phim cùng React" />
```

4. Lưu tệp, quan sát trình duyệt, sau đó thử bỏ prop `title`.

Kết quả mong đợi: tiêu đề nhận giá trị từ component cha; khi bỏ prop, trở về `Welcome`.

Tự kiểm tra: dấu `{}` trong JSX có vai trò gì? Giá trị mặc định của prop giúp ích ở đâu? Trong `MovieCard`, các trường `movie.title`, `movie.poster` và `movie.rating` được hiển thị thế nào?

6. Bài 3 — State, sự kiện và custom Hook

Mục tiêu: thấy quan hệ giữa thao tác người dùng, state và dữ liệu.

1. Mở `src/components/TrendingSection/TrendingSection.jsx`, tìm `useState('Today')` và lời gọi `setPeriod(item)`.

2. Ngay sau thẻ mở `<section className="trending page-width">`, thêm:

```
<p aria-live="polite">Đang xem: {period === 'Today' ? 'Hôm nay' :
'Tuần này'}</p>
```

3. Chuyển `Today` / `This Week`, quan sát nhãn vừa thêm.

4. Mở `src/hooks/useTrendingMovies.js`, tìm cách đổi `period` thành `day` hoặc `week` trong URL API.

Kết quả mong đợi: nhãn đổi theo lựa chọn. Khi có token, truy vấn dùng URL tương ứng ngày/tuần. Danh sách có thể chứa nhiều phim giống nhau; kiểm tra URL trong Network thay vì chỉ so tên phim.

Tự kiểm tra: nếu chỉ gán một biến thường, điều gì giúp React biết cần cập nhật giao diện? Vì sao tách phần gọi API vào custom Hook?

7. Bài 4 — Dữ liệu bất đồng bộ và cache

Mục tiêu: phân biệt state giao diện và dữ liệu từ máy chủ. Cần token đọc hợp lệ.

1. Mở Developer Tools → Network, lọc `api.themoviedb.org`.
2. Tải lại trang chủ, chuyển `Today` → `This Week` → `Today` trong vòng 5 phút.
3. Đối chiếu yêu cầu với `queryKey: ['trending-movies', period]` và `staleTime` trong `useTrendingMovies.js`.
4. Tìm một từ khóa, chuyển trang rồi quay lại. Đọc `queryKey`, `enabled`, `placeholderData` trong `useSearch.js`; giải thích vì sao khóa chứa loại nội dung, từ khóa và số trang.

Kết quả mong đợi: dữ liệu còn mới có thể được dùng lại khi quay về cùng khóa truy vấn. Số yêu cầu thực tế phụ thuộc cache đang có, yêu cầu thành công hay thất bại và cơ chế thử lại. Không mặc định một lần render luôn tương ứng một yêu cầu mạng. Tham khảo các mặc định của TanStack Query.

Mở rộng tùy chọn: đọc `useMovieActions.js` và `useWatchlist.js`, tìm `onMutate`, `onError`, `onSettled`. Mô tả cập nhật giao diện trước, khôi phục khi lỗi và làm mới dữ liệu. Chỉ thử ghi khi tài khoản của bạn có quyền phù hợp. Cache trang watchlist và thao tác trang chi tiết hiện dùng khóa khác nhau; việc chúng tự đồng bộ là vấn đề cần kiểm tra, chưa phải hành vi được bảo đảm.

Tự kiểm tra: `staleTime` khác thời gian lưu cache thế nào? Vì sao `isLoading` và `isFetching` có thể khác nhau? State `period` có cùng vai trò với danh sách phim không?

8. Bài 5 — Định tuyến và tải mã theo trang

Mục tiêu: hiểu URL lựa chọn màn hình và quan sát lazy loading.

1. Trong `AppRouter.jsx`, tìm `/`, `/search`, `/watchlist`, `/movies/:movieId`.
2. Mở thẻ phim từ trang chủ, đối chiếu ID trên URL với `useParams()` trong `MovieDetailsPage.jsx`.
3. Truy cập `/search?query=Batman&type=movie&page=1`, đọc cách `SearchPage.jsx` dùng `useSearchParams()`.
4. Đọc các lời gọi `lazy(() => import(...))` trong `Lazypages.jsx` và giao diện chờ của `Suspense` trong `AppRouter.jsx`.
5. Với bản build/preview ở phần 10, mở Network trước khi tải trang, bật `Disable cache`, tải lại trang chủ rồi điều hướng sang tìm kiếm. Quan sát tệp JavaScript được yêu cầu; chọn mạng chậm nếu muốn dễ thấy giao diện chờ.

Kết quả mong đợi: URL tương ứng màn hình; các trang được tổ chức qua dynamic import. Tên và số chunk do công cụ build quyết định, có thể có chunk dùng chung. Loader có thể không nhìn thấy khi mã tải quá nhanh.

Tự kiểm tra: tham số đường dẫn khác query string ở đâu? `Suspense` hiện tại chờ mã trang hay mọi yêu cầu API? Khi nào màn hình cần tự hiển thị trạng thái tải dữ liệu?

9. Bài 6 — Xử lý lỗi giao diện

Mục tiêu: phân biệt Error Boundary và lỗi API.

1. Trong `HomePage.jsx`, thêm hoặc bỏ chú thích dòng sau ở đầu hàm `HomePage`, trước `return`:

```
throw new Error('Lỗi thực hành Error Boundary')
```

2. Mở trang chủ. Nếu Vite hiện lớp báo lỗi phát triển, đóng lớp đó để quan sát bên dưới.
3. Đọc `getDerivedStateFromError`, `componentDidCatch`, `handleReset` trong `ErrorBoundary.jsx`.
4. Nhấn `Thử tải lại`: khi dòng `throw` còn, component tiếp tục lỗi sau khi Boundary reset. Phân biệt phục hồi trạng thái và sửa nguyên nhân lỗi.
5. Xóa dòng `throw`, lưu rồi tải lại trang. Không giữ lỗi mô phỏng khi chuyển bài.
6. Để so sánh lỗi dữ liệu, tạm để token trống, khởi động lại dev server, mở tìm kiếm có từ khóa. Quan sát thông báo tại trang, sau đó khôi phục token.

Kết quả mong đợi: lỗi render trong component con hiển thị giao diện thay thế; lỗi truy vấn được trang xử lý bằng trạng thái riêng. Error Boundary không tự bắt mọi lỗi trong event handler hay tác vụ bất đồng bộ. Xem phạm vi Error Boundary.

Tự kiểm tra: vì sao reset không sửa được lệnh `throw` luôn thực thi? Khi API thất bại, lỗi được đưa ra giao diện ở đâu?

10. Build và triển khai thử

10.1. Tạo và xem bản build

Gỡ lỗi mô phỏng, khôi phục cấu hình rồi chạy:

```
npm run build
npm run preview
```

Mở URL preview trong terminal, thường là `http://localhost:4173`. `build` tạo `dist`; `preview` giúp kiểm tra bản build tại máy, không dùng làm máy chủ production. Xem hướng dẫn triển khai Vite.

Cấu hình hiện tại còn tạo `stats.html` ở thư mục gốc và có thể tự mở trình duyệt. Mở tệp để xem treemap các module. Ở máy không có giao diện đồ họa, có thể đổi `open: true` thành `open: false` trong cấu hình `visualizer`.

Kiểm tra: trang chủ, URL trực tiếp `/search?query=Batman&type=movie&page=1`, mở chi tiết từ thẻ phim và tải lại tại trang chi tiết. Quan sát Network/Console nếu khác môi trường dev.

10.2. Đưa lên dịch vụ lưu trữ tĩnh

Dự án hiện là frontend gọi TMDB trực tiếp, không có backend riêng cần khởi động.

1. Chọn dịch vụ/máy chủ hỗ trợ website tĩnh và SPA fallback.
2. Nếu tự build từ kho mã: thư mục gốc chứa `package.json`, Node phù hợp, lệnh cài `npm ci`, lệnh build `npm run build`, thư mục xuất bản `dist`.
3. Khai báo biến môi trường trước build. Nếu build tại máy cá nhân, tải **nội dung `dist`** lên vùng phục vụ website.
4. Cấu hình URL ứng dụng như `/search`, `/movies/123` trả về `index.html` khi không trỏ tới tệp tĩnh có thật. Thiếu SPA fallback, tải lại URL con có thể báo 404.

5. Kiểm tra lại các đường dẫn và trạng thái lỗi trên địa chỉ đã triển khai.

Hướng dẫn mặc định chạy ở gốc tên miền `/`. Đưa vào thư mục con còn cần đồng bộ `base` của Vite, `basename` của Router và liên kết tuyệt đối. Thay biến `VITE_` cần build và triển khai lại; sửa `.env` sau build không đổi mã trong `dist`.

11. Bài nâng cao — So sánh React Compiler

Mục tiêu: thực hành đo và trình bày đúng phạm vi. Có thể học các bài trước mà chưa làm phần này.

Đọc `process.env.REACT_COMPILER !== 'off'` trong `vite.config.js`: mặc định bật Compiler, đổi cấu hình cần khởi động lại Vite.

macOS/Linux, chạy riêng từng cấu hình, dừng tiến trình trước khi chuyển:

```
REACT_COMPILER=off npm run dev
```

```
REACT_COMPILER=on npm run dev
```

Windows PowerShell:

```
$env:REACT_COMPILER = 'off'
npm run dev
```

Dừng bằng `Ctrl+C`, sau đó:

```
$env:REACT_COMPILER = 'on'
npm run dev
```

Xác nhận terminal in `[React Compiler] OFF`/`ON`. Biến PowerShell giữ giá trị trong phiên terminal; dùng `Remove-Item Env:REACT_COMPILER` để trở về mặc định khi xong.

Các bước đo:

1. Cài React Developer Tools, mở Profiler. Giữ cùng máy, trình duyệt, dữ liệu và thứ tự thao tác ở cả hai cấu hình.
2. Chờ truy vấn ban đầu hoàn tất, chuyển Today/This Week một lượt để làm nóng cache rồi về Today. Bắt đầu ghi, thực hiện cùng một chuỗi chuyển nút và dừng.
3. Xuất JSON, lặp lại, ví dụ 5 phiên mỗi cấu hình. Ghi số commit thực tế, không giả định chúng luôn bằng nhau.
4. Để đo build, thay `npm run dev` trong các lệnh đặt biến bằng `npm run build`. Ghi thời gian `built in`, kích thước JavaScript, gzip; lặp lại cùng số lần.

5. Lập bảng gồm cấu hình, môi trường, thao tác, số phiên, số commit, thời gian render và build. Thời gian render trung bình/commit = tổng duration chia tổng số commit của dữ liệu tương ứng.

Đối chiếu cách diễn giải với báo cáo Profiler và báo cáo build. Đây là dữ liệu mẫu của nhóm; máy của bạn không cần cho ra đúng con số đó. Profiler có chi phí ghi nhận và ứng dụng dùng StrictMode; ghi rõ điều kiện đo.

Kết quả cần nộp: bảng số liệu của bạn, nhận xét về lợi ích, chi phí và giới hạn. Thời gian render khác thời gian tải trang; giảm duration không tự chứng minh giảm số commit hay tăng tốc toàn bộ ứng dụng.

12. Kiểm tra mã và xử lý vướng mắc

12.1. Các lệnh kiểm tra

```
npm run lint
npm run test -- --run
```

`--run` yêu cầu Vitest chạy một lần. Các lệnh giúp phát hiện vấn đề; tài liệu không khẳng định kho mã đã vượt qua mọi kiểm tra.

Test hiện có: Error Boundary hiển thị `Header`, mà `Header` cần Router. Đọc mã cho thấy test hiện chưa bọc Router, nên có thể phát sinh lỗi thiếu Router context khi hiển thị fallback. Để thực hành sửa, thêm import vào `src/components/ErrorBoundary/ErrorBoundary.test.jsx`:

```
import { MemoryRouter } from 'react-router-dom'
```

Thay lời gọi `render` trong test bằng:

```
render(
  <MemoryRouter>
    <ErrorBoundary>
      <ThrowError />
    </ErrorBoundary>
  </MemoryRouter>
)
```

Chạy lại test. Đây là bước thay đổi dành cho người thực hành; việc tạo tài liệu không tự sửa tệp kiểm thử.

12.2. Lỗi thường gặp

Hiện tượng	Cách kiểm tra/xử lý
Không nhận `node`/`npm`	Cài Node, mở lại terminal, kiểm tra phiên bản
`EBADENGINE` hoặc lỗi công cụ trên Node cũ	Đối chiếu phần 2, cài bản phù hợp rồi chạy lại `npm ci`

Hiện tượng	Cách kiểm tra/xử lý
Lockfile không khớp	Kiểm tra `package.json` và lockfile cùng phiên bản kho mã; trao đổi với nhóm nếu sai lệch
Không tải được dependency	Kiểm tra Internet, proxy và quyền truy cập npm registry
Thiếu TMDB token	Kiểm tra vị trí `.env`, tên biến rồi khởi động lại Vite
API trả 401/403	Kiểm tra loại token; với chức năng tài khoản, kiểm tra quyền và xác thực người dùng
Có Trending nhưng không tìm được phim	Có thể đang dùng dữ liệu mẫu do token trống
Thêm watchlist thất bại	Không dùng ID mặc định/chuỗi ví dụ; xem xác thực ở phần 3
Danh sách chưa đổi ở màn hình khác	Kiểm tra query key và invalidate giữa các Hook
Reset vẫn lỗi mô phỏng	Xóa dòng `throw`, lưu, tải lại
Menu mở trang đang phát triển	Chức năng đó chưa hoàn chỉnh
URL con báo 404 trên hosting	Cấu hình SPA fallback về `index.html`
Sửa `.env` nhưng bản build không đổi	Build lại, tải bản mới lên hosting
Không thấy tải JavaScript khi chuyển trang	Mở Network trước khi tải, tắt cache, thử bản preview

13. Kết quả thực hành và tự đánh giá

Hoàn thành phần cơ bản khi bạn có thể:

- Chạy ứng dụng và giải thích cấu hình cần thiết.
- Thay tiêu đề bằng props, quan sát nhãn thay đổi theo state.
- Chỉ ra luồng component → custom Hook → API → giao diện.
- Phân biệt state cục bộ, dữ liệu truy vấn và dữ liệu lưu trên trình duyệt.
- Giải thích đường dẫn, query string, lazy loading và trạng thái chờ dữ liệu.
- Mô phỏng rồi gỡ lỗi render, phân biệt Error Boundary với lỗi truy vấn.
- Build, xem bằng preview và nêu yêu cầu SPA fallback.

Lưu ảnh minh chứng, đoạn mã thay đổi và câu trả lời tự kiểm tra. Với bài Compiler, bổ sung JSON, bảng số liệu và điều kiện đo. Không đưa token hay thông tin xác thực vào ảnh và bài nộp.

Bài mở rộng: thêm bộ lọc giao diện, liên kết kết quả tìm kiếm đến chi tiết hoặc sửa đồng bộ cache watchlist. Với mỗi thay đổi, mô tả nhu cầu, tệp liên quan, cách kiểm tra và nội dung React đã vận dụng.